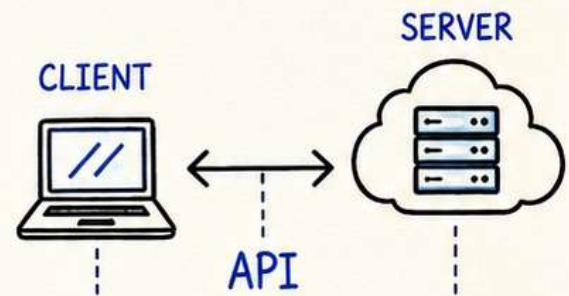


REST API Interview Questions & Answers

1. What is an API?

Answer:

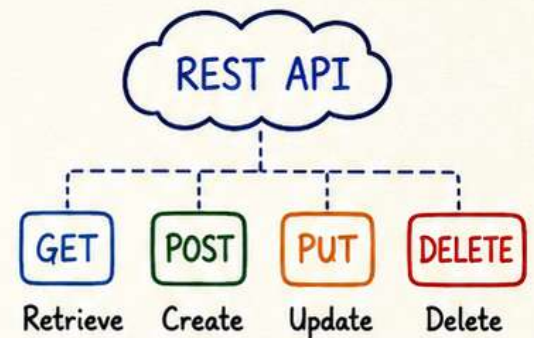
- API stands for **Application Programming Interface**.
- It is a set of rules that allows two software applications to communicate with each other.
- APIs act as a **bridge** between client and server.
- Examples: **REST API**, **SOAP API**, **GraphQL API**.



2. What is a REST API?

Answer:

- REST stands for **Representational State Transfer**.
- It is an architectural style for designing networked applications.
- It uses **HTTP** protocol to perform **CRUD** operations.
- It is **stateless**, **scalable** and easy to maintain.



3. What are REST principles?

Answer:

- **Client-Server**: Separation of concerns.
- **Stateless**: Server does not store client state.
- **Cacheable**: Responses can be cached to improve performance.
- **Uniform Interface**: Consistent and standard interactions.
- **Layered System**: Multiple layers for security and scalability.
- **Code on Demand** (Optional): Server can extend client functionality.



4. What are the advantages of REST?

Answer:

- **Simple** and easy to understand.
- Uses standard **HTTP** methods.
- **Lightweight** and high performance.
- **Scalable** and flexible.

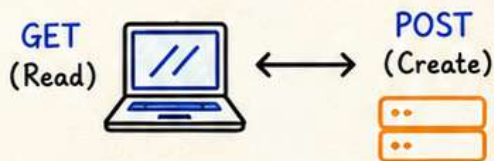


REST API Interview Questions & Answers

5. What is the difference between GET and POST?

Answer:

- **GET** is used to retrieve data from the server. It should not change server state.
- **POST** is used to create new data on the server. It can change the server state.
- **GET** is idempotent and safe.
- **POST** is not idempotent.



6. PUT vs PATCH?

Answer:

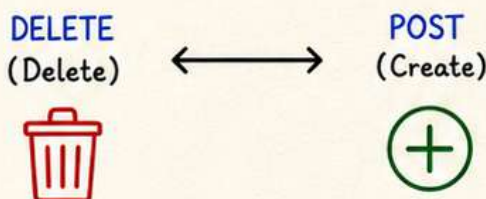
- **PUT** is used to update the entire resource. It replaces the resource completely.
- **PATCH** is used to update part of a resource. It modifies only the specified fields.
- **PUT** is idempotent; **PATCH** is also idempotent.



7. DELETE vs POST?

Answer:

- **DELETE** is used to remove a resource from the server.
- **POST** is used to create a new resource.
- **DELETE** should be idempotent.
- **POST** is not idempotent.



8. Which HTTP methods are idempotent?

Answer:

- **GET, PUT, DELETE** are idempotent.
- Idempotent means multiple identical requests have the same effect as one.
- **POST** and **PATCH** are not idempotent.



Quick Tip

Idempotent methods can be retried without causing an unintended effect.



REST API Interview Questions & Answers

5. What is the difference between GET and POST?

Answer:

- GET is used to retrieve data from the server.
- POST is used to send data to the server to create a new resource.
- GET is idempotent and safe.
- POST is not idempotent.
- GET data is visible in URL. POST data is in request body.

GET → Retrieve Data 

Example: GET /users/123
→ Get user with ID 123

POST → Send Data / Create 

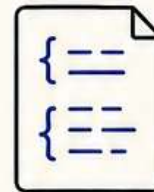
Example: POST /users
→ Create a new user

6. What is the difference between PUT and PATCH?

Answer:

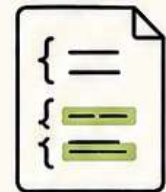
- PUT is used to update or replace the entire resource.
- PATCH is used to update only partial data of a resource.
- PUT can create the resource if it doesn't exist.
- PATCH only modifies the provided fields.

PUT
Replace Entire Resource



Sends all fields of the resource.

PATCH
Update Partial Resource



Sends only the fields to be updated.

7. What is the difference between DELETE and POST?

Answer:

- DELETE is used to remove a resource.
- POST is used to create a resource or perform an action.
- DELETE is idempotent.
- POST is not idempotent.

DELETE → Remove Resource 

Example: DELETE /users/123
→ Delete user with ID 123

POST → Create / Action 

Example: POST /users
→ Create a new user

8. Which HTTP methods are idempotent?

Answer:

- Idempotent methods return the same result even if the request is repeated.
- Idempotent methods: GET, PUT, DELETE, HEAD, OPTIONS.
- POST is not idempotent.

Idempotent = Same Result no matter how many times you call it. 

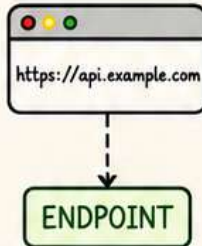


REST API Interview Questions & Answers

9. What is an endpoint?

Answer:

- An endpoint is a specific URL where an API can be accessed.
- It acts as the entry point to perform an operation on a resource.
- Each endpoint corresponds to a resource and an HTTP method.
- Example:



GET	/users	→ Get all users
POST	/users	→ Create user
GET	/users/101	→ Get user by ID
DELETE	/users/101	→ Delete user

10. What is a resource?

Answer:

- A resource is an object or data that can be accessed via an API.
- Resources are identified by URIs.
- They can be created, read, updated or deleted.
- Examples:



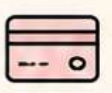
Users



Orders



Products



Payments

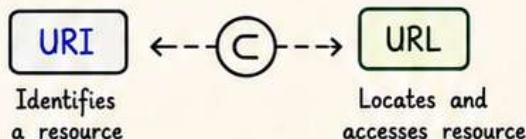


Everything in REST API is a resource. Even collections are resources.

11. What is the difference between URI and URL?

Answer:

- URI (Uniform Resource Identifier) is a unique identifier for a resource.
- URL (Uniform Resource Locator) is the address of the resource and tells how to access it.
- All URLs are URIs, but all URIs are not URLs.



Example:

URI:	/users/101
URL:	https://api.example.com/users/101

12. What is the difference between Path Parameters and Query Parameters?

Answer:

• Path Parameters:

- Used to identify a specific resource.
- Part of the URL path.
- Example: /users/101

/users/101
↑
Path Parameter

• Query Parameters:

- Used to filter, search, sort, or customize the response.
- Appended after ? in the URL.
- Example: /users?role=admin&age=25

/users?role=admin&age=25
↑
Query Parameters



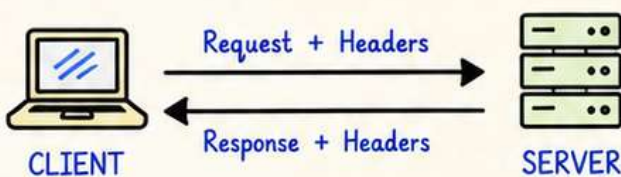
Path Parameters = Identify the resource
Query Parameters = Modify the result

REST API Interview Questions & Answers

13. What are HTTP headers?

Answer:

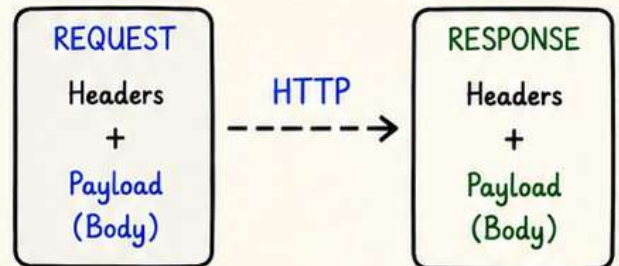
- Headers provide **metadata** about the request or response.
- They are **key-value** pairs.
- Common headers:
 - **Content-Type**: Type of data being sent.
 - **Authorization**: Auth token or credentials.
 - **Accept**: Type of response expected.
 - **Cache-Control**: Caching instructions.
 - **User-Agent**: Client information.



14. What is Payload?

Answer:

- Payload is the **body** of the request or response.
- It contains the **actual data**.
- Sent in **POST, PUT, PATCH** requests.
- Usually in **JSON** or **XML** format.



15. What is JSON?

Answer:

- **JSON** (JavaScript Object Notation) is a lightweight data format.
- Easy to read and write.
- Language independent.
- Widely used in **REST APIs**.



16. What is XML?

Answer:

- **XML** (eXtensible Markup Language) is a markup language.
- More verbose than JSON.
- Platform and language independent.
- Used in older APIs and enterprise systems.



Quick Tip
Headers = Metadata
Payload = Actual Data



JSON is preferred in modern REST APIs because it is lightweight and human readable.



REST API Interview Questions & Answers

17. What are HTTP status codes?

Answer:

- Status codes are standard responses from the server.
- They indicate the result of an API request.
- They are grouped into 5 classes:

1xx	Informational	Ex: 100
2xx	Success	Ex: 200, 201
3xx	Redirection	Ex: 301, 302
4xx	Client Error	Ex: 400, 404
5xx	Server Error	Ex: 500, 503

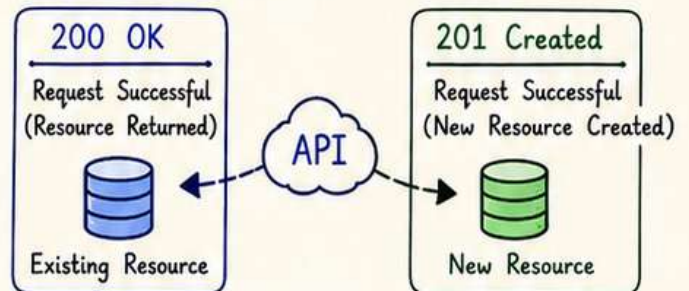


Status codes help clients understand what happened with their request.

18. What is the difference between 200 OK and 201 Created?

Answer:

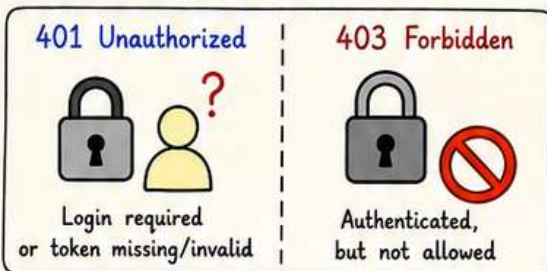
- **200 OK:** The request was successful and the resource is returned.
- **201 Created:** The request was successful and a new resource is created.
- 200 is usually for **GET, PUT, PATCH**.
- 201 is usually for **POST** when a new resource is created.



19. What is the difference between 401 Unauthorized and 403 Forbidden?

Answer:

- **401 Unauthorized:** Authentication is required or has failed.
- **403 Forbidden:** Authenticated but not allowed to access the resource.
- 401 → "Who are you?"
- 403 → "You are authenticated, but you don't have permission."



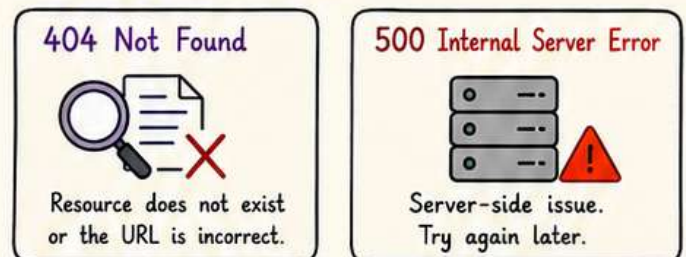
Example:

- 401 → No token / invalid token
- 403 → Valid token, but insufficient role or permission

20. What is the difference between 404 Not Found and 500 Internal Server Error?

Answer:

- **404 Not Found:** The requested resource does not exist on the server.
- **500 Internal Server Error:** Something went wrong on the server while processing the request.
- 404 is a client error.
- 500 is a server error.



Tip: Use appropriate status codes to provide clear feedback to API clients and help with debugging.

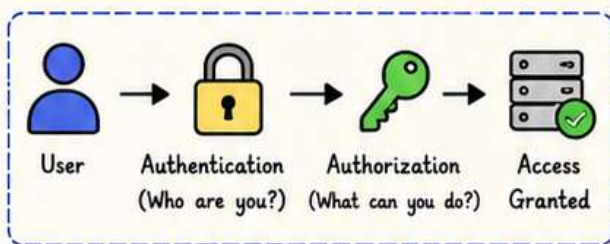


REST API Interview Questions & Answers

21. What is the difference between Authentication and Authorization?

Answer:

- **Authentication** verifies who you are.
- **Authorization** determines what you are allowed to do.
- Authentication happens first, then Authorization.

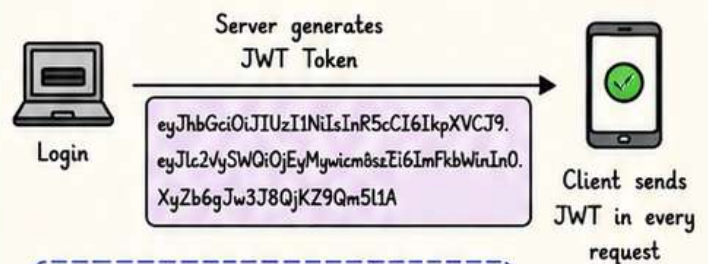


Think: Login → **Authentication**
Access to features → **Authorization**

22. What is JWT?

Answer:

- **JWT (JSON Web Token)** is a compact token used for authentication.
- It is **self-contained** and secure.
- Consists of **Header, Payload** and **Signature**.
- Commonly used in **stateless** REST APIs.



Benefits:

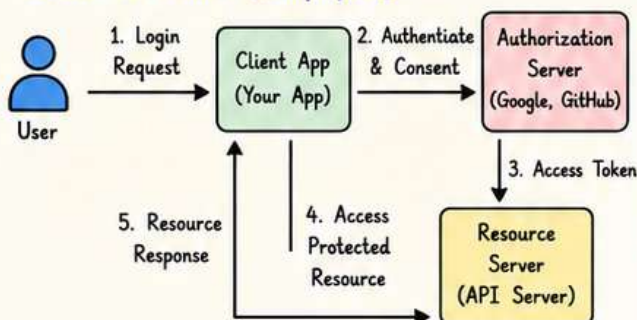
- ✓ Stateless
- ✓ Compact
- ✓ Secure (Digitally Signed)
- ✓ Can be used across multiple services

23. What is OAuth 2.0?

Answer:

- **OAuth 2.0** is an authorization framework.
- Allows **third-party** applications to access resources without sharing credentials.
- Commonly used for **delegated access**.
- Example: Login with Google, GitHub, etc.

OAuth 2.0 Flow (Simplified):



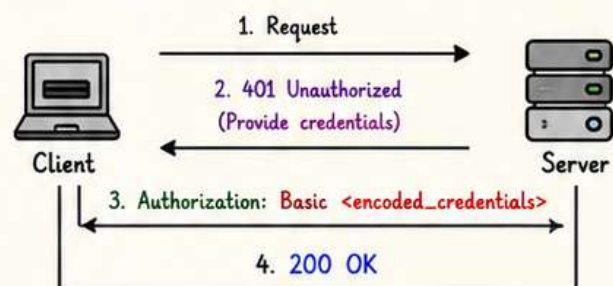
OAuth 2.0 = **Authorization** (Access Delegation)
Not Authentication

24. What is Basic Authentication?

Answer:

- Basic Authentication sends username and password in **Base64** encoding.
- Sent in the **Authorization** header.
- Simple but less secure (use over **HTTPS** only).
- Example:

Authorization: Basic `dXNlcm5hbWU6cGFzc3dwcmQ=`
(Base64 encoded username:password)



Always use **HTTPS** to protect credentials and tokens.

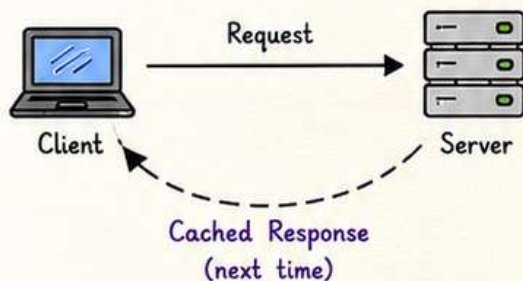


REST API Interview Questions & Answers

25. What is Caching in REST APIs?

Answer:

- Caching stores responses so that they can be reused for future requests.
- Improves performance and reduces server load.
- Implemented using headers like Cache-Control, ETag, Expires.
- Can be done on client-side, server-side or CDN.

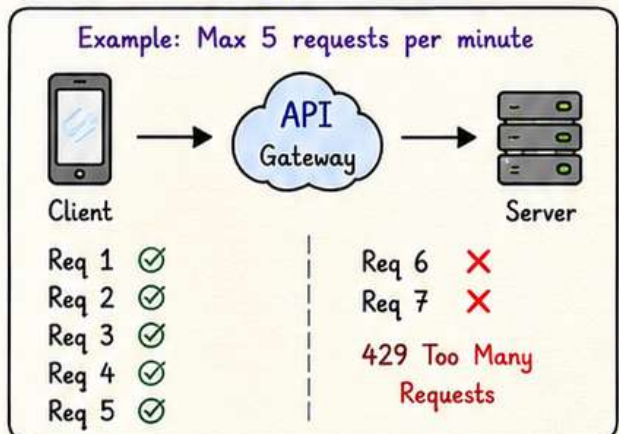


Tip: Use Cache-Control and ETag headers for smart caching.

26. What is Rate Limiting?

Answer:

- Rate Limiting restricts the number of requests a client can make in a given time.
- Prevents abuse, DDoS attacks and ensures fair usage.
- Implemented using counters, sliding window, token bucket, leaky bucket algorithms.



HTTP Status 429 (Too Many Requests) is used when rate limit is exceeded.

27. What is API Versioning?

Answer:

- Versioning helps manage changes in APIs without breaking existing clients.
- Multiple versions can coexist.
- Common strategies:
 - ① URI Versioning: /v1/users
 - ② Header Versioning: Accept-Version: v1
 - ③ Query Versioning: /users?version=1
 - ④ Media Type Versioning: Accept: application/vnd.app.v1+json

Example (URI Versioning)

v1	v2
/v1/users	/v2/users
Old Response (compatible)	New Response (enhanced)



Tip: Deprecate old versions gracefully and communicate changes clearly.

28. What is Pagination?

Answer:

- Pagination divides large result sets into smaller chunks (pages).
- Improves performance and user experience.
- Common approaches:
 - ★ Page & Size (Offset-based)
 - ★ Cursor-based (using next cursor)
 - ★ Limit & Offset

Example (Page & Size)

GET /users?page=2&size=10

Response:

```
{
  "data": [ ... 10 users ... ],
  "page": 2,
  "size": 10,
  "totalPages": 25,
  "totalElements": 245
}
```



10 items
per page



Tip: Use cursor-based pagination for better performance with large datasets.

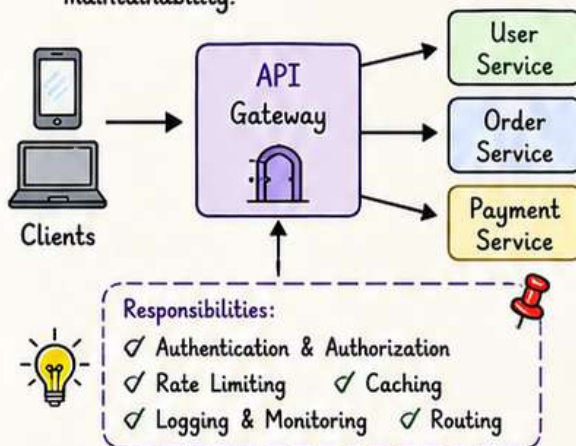


REST API Interview Questions & Answers

29. What is an API Gateway?

Answer:

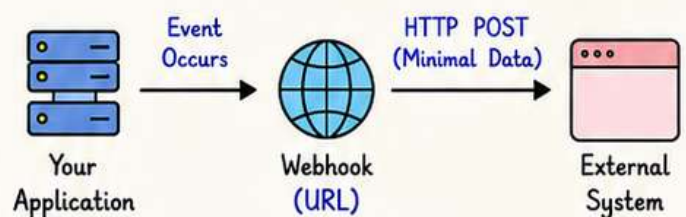
- API Gateway is a **single entry point** for all client requests.
- It routes requests to appropriate microservices.
- Handles **cross-cutting concerns** like authentication, rate limiting, logging, caching, etc.
- Improves **security, scalability** and maintainability.



30. What are Webhooks?

Answer:

- Webhooks are user-defined HTTP callbacks.
- Instead of polling, the server sends **real-time notifications** to the client.
- Commonly used for events like payments, order placed, user created, etc.
- More efficient and reduces server load.



Example:

When a new order is created → your server sends a **POST** request to the webhook URL of the payment provider.

Tip: Webhooks enable real-time communication without constant polling.

31. What is HATEOAS?

Answer:

- HATEOAS stands for **Hypermedia As The Engine Of Application State**.
- API responses include **links** to related actions.
- Clients discover actions dynamically via links.
- Improves flexibility and decouples client and server.

Example Response:

```
{
  "id": 101,
  "name": "John Doe",
  "email": "john@example.com",
  "links": [
    { "rel": "self", "href": "/users/101" },
    { "rel": "update", "href": "/users/101" },
    { "rel": "delete", "href": "/users/101" },
    { "rel": "orders", "href": "/users/101/orders" }
  ]
}
```

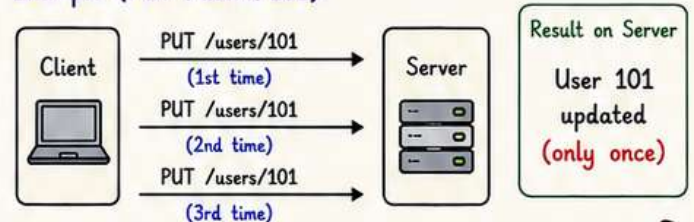
Tip: HATEOAS makes APIs more discoverable and reduces hard-coded URLs in clients.

32. What is Idempotency?

Answer:

- An idempotent operation produces the same result even if repeated multiple times.
- Important for reliability (e.g., retries due to network failure).
- HTTP methods that are idempotent: **GET, PUT, DELETE, HEAD, OPTIONS**.
- **POST** is not idempotent by default.

Example (PUT /users/101):



Tip: Use Idempotency-Key header with POST requests to achieve idempotency.

Example:

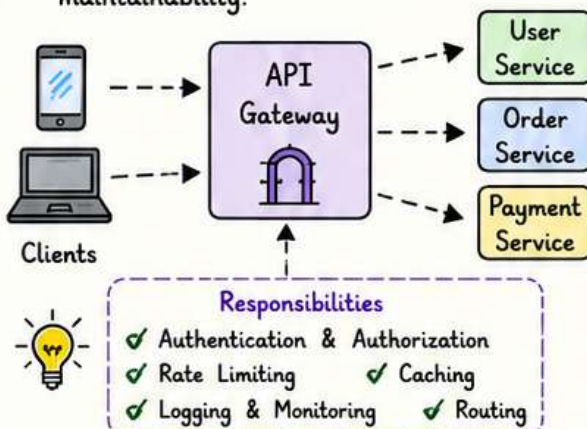
Idempotency-Key: 7f8e2b9d-4c3a-4f9b-bb9d-123456789abc

REST API Interview Questions & Answers

29. What is an API Gateway?

Answer:

- API Gateway is a **single entry point** for all client requests.
- It routes requests to appropriate **microservices**.
- Handles **cross-cutting concerns** like auth, rate limiting, logging, caching, etc.
- Improves **security, scalability** and maintainability.

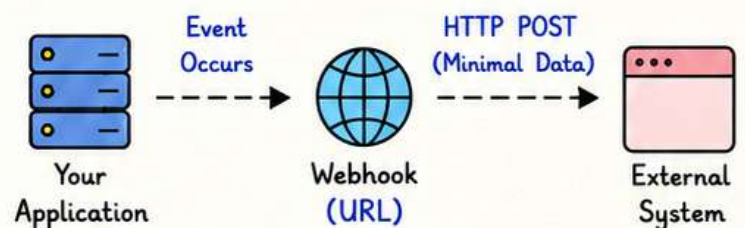


Tip: Think of API Gateway as a smart traffic controller for APIs.

30. What are Webhooks?

Answer:

- Webhooks are user-defined HTTP callbacks.
- Instead of polling, the server sends **real-time notifications** to the client.
- Commonly used for events like payments, order placed, user created, etc.
- More efficient and reduces server load.



Example:

When a new order is created → your server sends a **POST** request to the webhook URL of the payment provider.

Tip: Webhooks enable real-time communication without constant polling.

31. What is HATEOAS?

Answer:

- HATEOAS stands for **Hypermedia As The Engine Of Application State**.
- API responses include **links** to related actions.
- Clients discover actions dynamically via links.
- Improves flexibility and decouples client and server.

Example Response:

```

{
  "id": 101,
  "name": "John Doe",
  "email": "john@example.com",
  "links": [
    { "rel": "self", "href": "/users/101" },
    { "rel": "update", "href": "/users/101" },
    { "rel": "delete", "href": "/users/101" },
    { "rel": "orders", "href": "/users/101/orders" }
  ]
}
  
```

**Links Guide
The Client**

- View →
- Update →
- Delete →
- Orders →

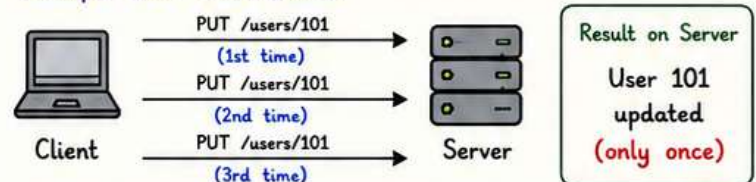
Tip: HATEOAS makes APIs more discoverable and reduces hard-coded URLs in clients.

32. What is Idempotency?

Answer:

- An idempotent operation produces the **same result** even if repeated multiple times.
- Important for reliability (e.g., retries due to network failure).
- HTTP methods that are idempotent: **GET, PUT, DELETE, HEAD, OPTIONS**.
- **POST** is not idempotent by default.

Example (PUT /users/101):



Tip: Use Idempotency-Key header with POST requests to achieve idempotency.

Example:

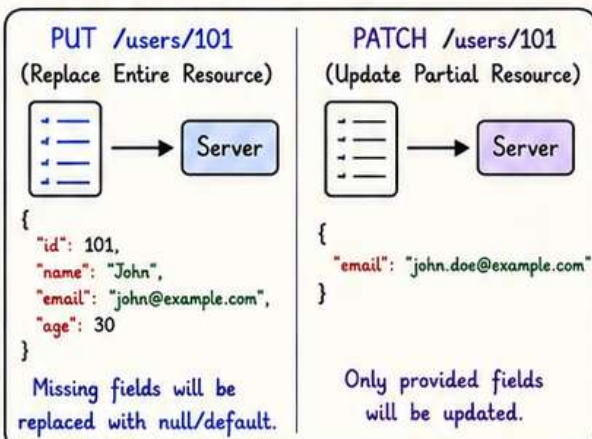
Idempotency-Key: 7f8e2b9d-4c3a-4f9b-bb9d-123456789abc

REST API Interview Questions & Answers

33. What is PUT vs PATCH?

Answer:

- **PUT** replaces the entire resource.
- **PATCH** updates only the specified fields.
- **PUT** is idempotent.
- **PATCH** can be idempotent or non-idempotent depending on implementation.

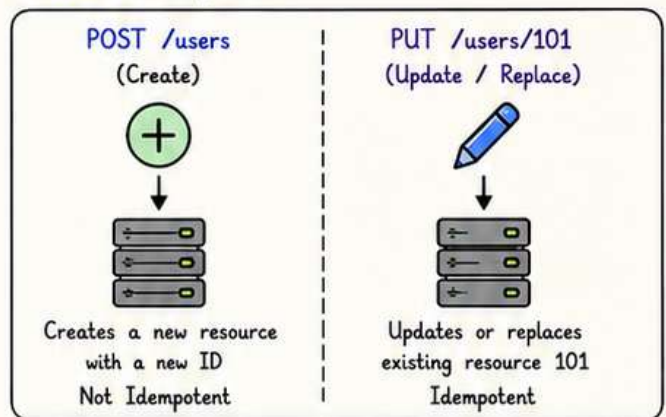


Tip: Use **PUT** when you want to replace.
Use **PATCH** when you want to update partially.

34. What is the difference between PUT and POST?

Answer:

- **POST** is used to create a new resource.
- **PUT** is used to update or replace an existing resource.
- **POST** is not idempotent.
- **PUT** is idempotent.



Example:

- **POST /users** → creates a new user (id: 102)
- **PUT /users/101** → updates or replaces user 101 each time (same result).

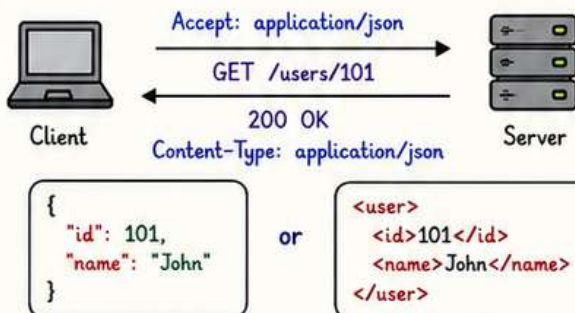


Tip: **POST** → Create | **PUT** → Update/Replace

35. What is Content Negotiation?

Answer:

- Content Negotiation is the mechanism to select the best representation of a resource.
- It is done using the **Accept** header.
- Based on client preference, server returns JSON, XML, etc.

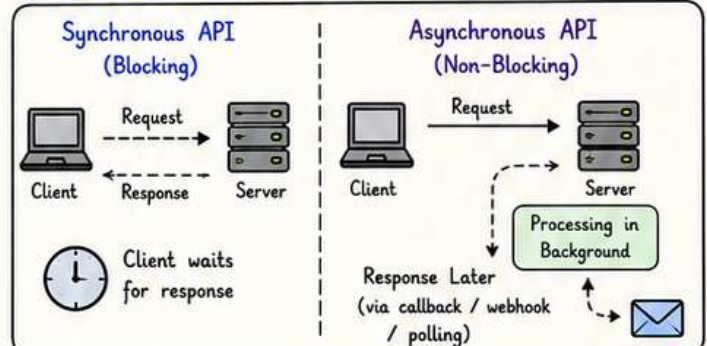


Tip: Server checks Accept, Accept-Language, Content-Type headers to respond appropriately.

36. What is the difference between Synchronous and Asynchronous APIs?

Answer:

- **Synchronous:** Client waits for the response.
- **Asynchronous:** Client doesn't wait for the response; processing happens in background.
- Async is better for long-running tasks.



Tip: Use Async APIs for emails, payments, file processing, notifications, etc.



Key Takeaway: Understand concepts deeply and relate them with real-world scenarios.
Design clean, scalable and secure REST APIs.

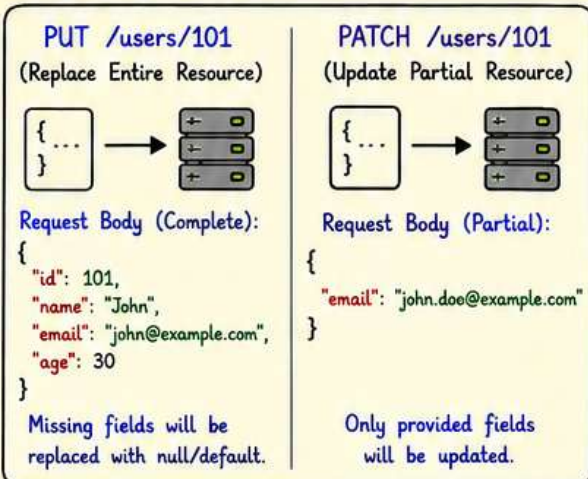


REST API Interview Questions & Answers

33. What is PUT vs PATCH?

Answer:

- **PUT** replaces the entire resource.
- **PATCH** updates only the specified fields.
- **PUT** is idempotent.
- **PATCH** can be idempotent or non-idempotent depending on implementation.

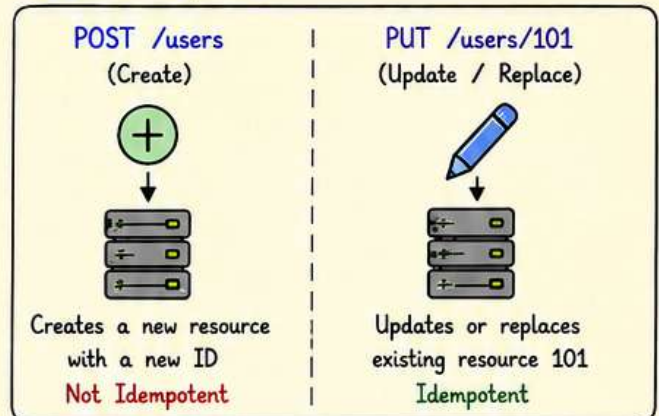


Tip: Use **PUT** when you want to replace the resource completely.
Use **PATCH** when you want to update partially.

34. What is the difference between PUT and POST?

Answer:

- **POST** is used to create a new resource.
- **PUT** is used to update or replace an existing resource.
- **POST** is not idempotent.
- **PUT** is idempotent.



Example:

- **POST /users** → creates a new user (id: 102)
- **PUT /users/101** → updates or replaces user 101 each time (same result).

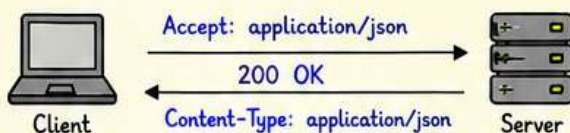


Tip: **POST** → Create | **PUT** → Update/Replace

35. What is Content Negotiation?

Answer:

- Content Negotiation is the mechanism to select the best representation of a resource.
- It is done using the **Accept** header.
- Based on client preference, server returns JSON, XML, HTML, etc.



Request Header:
Accept: application/json
Response Body (JSON):

```
{
  "id": 101,
  "name": "John"
}
```

or

Request Header:
Accept: application/xml
Response Body (XML):

```
<user>
  <id>101</id>
  <name>John</name>
</user>
```

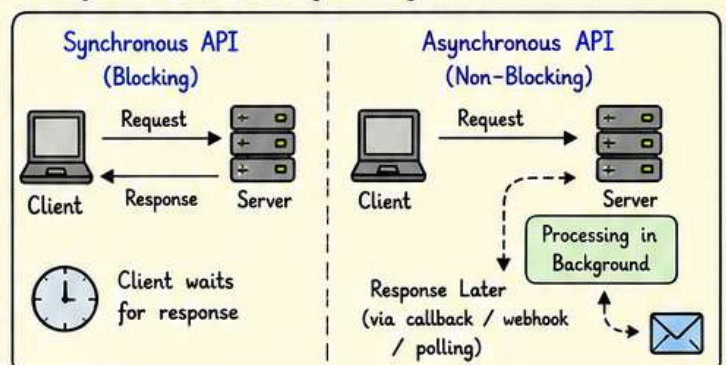


Tip: Server checks **Accept**, **Accept-Language**, **Content-Type** headers to respond appropriately.

36. What is the difference between Synchronous and Asynchronous APIs?

Answer:

- **Synchronous:** Client waits for the response.
- **Asynchronous:** Client doesn't wait for the response; processing happens in background.
- Async is better for long-running tasks.



Tip: Use Async APIs for emails, payments, file processing, notifications, etc.



Key Takeaway: Understand concepts deeply and relate them with real-world scenarios.
Design clean, scalable and secure REST APIs.



REST API Interview Questions & Answers

37. What are Best Practices for Designing REST APIs?

Answer:

- Use **nouns**, not verbs in URIs.
- Keep URIs **short, clear** and consistent.
- Use plural nouns: **/users** not **/user**.
- Use proper **HTTP methods**.
- Use **query parameters** for filtering, sorting, pagination.
- Use meaningful **status codes**.
- **Version** your APIs.
- **Secure** your APIs (Auth, HTTPS, input validation).
- Provide helpful **error messages**.
- Keep responses **consistent**.

Good vs Bad Examples

✓ Good	✗ Bad
/users	/getUsers
/users/101	/getUserById?id=101
/users?page=2&size=10	/usersPageTwo
/orders/101/items	/orderItemsOfOrder101
Accept: application/json	(no Accept header)



Tip: Follow REST principles, keep it simple and easy to understand for everyone.

38. How do you Handle Errors in REST APIs?

Answer:

- Use appropriate **HTTP status codes**.
- Return a **consistent error response** structure.
- Include **meaningful error message** and details.

Example Error Response (JSON)

```
{
  "timestamp": "2025-05-25T10:30:00Z",
  "status": 404,
  "error": "Not Found",
  "message": "User with ID 101 not found",
  "path": "/users/101"
}
```



Tip: Never expose stack traces or internal server details to clients.

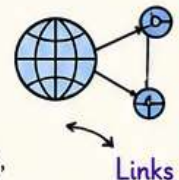
39. What is HATEOAS?

Answer:

- HATEOAS stands for **Hypermedia As The Engine Of Application State**.
- API responses include links to related actions.
- Clients discover actions dynamically via links.
- Improves flexibility and decouples client and server.

HATEOAS Example Response

```
{
  "id": 101,
  "name": "John Doe",
  "email": "john@example.com",
  "links": [
    { "rel": "self", "href": "/users/101" },
    { "rel": "update", "href": "/users/101" },
    { "rel": "delete", "href": "/users/101" },
    { "rel": "orders", "href": "/users/101/orders" }
  ]
}
```

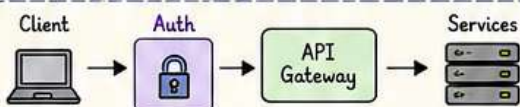


Tip: HATEOAS makes APIs more discoverable and easier to evolve.

40. How do you Secure REST APIs?

Answer:

- Use **HTTPS** for secure communication.
- **Authenticate** users (JWT / OAuth 2.0 / API Keys).
- **Authorize** access using roles / permissions.
- **Validate** and sanitize all input.
- Implement **rate limiting** and throttling.
- Log and **monitor** API usage.



Tip: Security is not optional. Always follow the principle of least privilege.

Bonus: Common REST API Interview Follow-up Questions

- ✓ Why is REST stateless?
- ✓ What are the limitations of REST?
- ✓ How does REST handle file uploads?
- ✓ What is API rate limiting and how to implement it?
- ✓ What is the difference between REST and SOAP?
- ✓ How would you design a search API?
- ✓ How do you version a REST API?
- ✓ How do you handle pagination in large datasets?



Key Takeaway: REST is all about simplicity, scalability and standardization. Focus on good design, security and a great developer experience.

